Código-fonte de referência

Versão final da implementação. As quebras de linhas longas são apenas editoriais; os arquivos originais em src são a referência executável. Cada linha original começa com seu número. O guia do código explica as responsabilidades e os métodos.

src/main/java/br/edu/catalogo/config/BancoDados.java

```
001 | package br.edu.catalogo.config;
002 |
003 | import br.edu.catalogo.exception.PersistenciaException;
004 | import java.io.IOException;
005 | import java.nio.charset.StandardCharsets;
006 | import java.sql.SQLException;
007 |
008 | /** Cria apenas estruturas ausentes, sem apagar ou popular dados do usuário. */
009 | public final class BancoDados {
010 |     private BancoDados() { }
011 |     public static void inicializar(FabricaConexao fabrica) {
012 |         try (var recurso = BancoDados.class.getResourceAsStream("/db/schema.sql")) {
013 |             if (recurso == null) throw new IllegalStateException("schema.sql não
encontrado.");
014 |             String script = new String(recurso.readAllBytes(), StandardCharsets.UTF_8);
015 |             try (var conexao = fabrica.abrir(); var comando = conexao.createStatement()) {
016 |                 for (String sql : script.split(";")) {
017 |                     if (!sql.isBlank()) comando.execute(sql);
018 |                 }
019 |             }
020 |         } catch (SQLException | IOException e) {
021 |             throw new PersistenciaException("inicializar a estrutura", e);
022 |         }
023 |     }
024 | }
```

src/main/java/br/edu/catalogo/config/ConexaoJdbc.java

```
001 | package br.edu.catalogo.config;
002 |
003 | import java.sql.Connection;
004 | import java.sql.DriverManager;
005 | import java.sql.SQLException;
006 |
007 | /** Centraliza URL e credenciais; cada operação DAO fecha sua própria conexão. */
008 | public final class ConexaoJdbc implements FabricaConexao {
009 |     private final String url;
010 |     private final String usuario;
011 |     private final String senha;
012 |
013 |     public ConexaoJdbc(String url, String usuario, String senha) {
014 |         this.url = url;
015 |         this.usuario = usuario;
016 |         this.senha = senha;
017 |         try { Class.forName("org.h2.Driver"); }
018 |         catch (ClassNotFoundException e) { throw new IllegalStateException("Driver H2
indisponível.", e); }
019 |     }
020 |     public static ConexaoJdbc doAmbiente() {
021 |         return new ConexaoJdbc(valor("CATALOGO_DB_URL", "jdbc:h2:file:./data/catalogo"),
022 |                 valor("CATALOGO_DB_USER", "sa"), valor("CATALOGO_DB_PASSWORD", ""));
023 |     }
024 |     private static String valor(String nome, String padrao) {
025 |         String valor = System.getenv(nome);
026 |         return valor == null ? padrao : valor;
027 |     }
028 |     @Override public Connection abrir() throws SQLException {
029 |         return DriverManager.getConnection(url, usuario, senha);
030 |     }
031 | }
```

src/main/java/br/edu/catalogo/config/FabricaConexao.java

```
001 | package br.edu.catalogo.config;
002 |
003 | import java.sql.Connection;
004 | import java.sql.SQLException;
005 |
006 | /** Contrato injetável: produção usa arquivo H2 e testes usam banco isolado. */
007 | @FunctionalInterface
008 | public interface FabricaConexao {
009 |     Connection abrir() throws SQLException;
010 | }
```

src/main/java/br/edu/catalogo/dao/ItemMidiaDao.java

```
001 | package br.edu.catalogo.dao;
002 |
003 | import br.edu.catalogo.model.ItemMidia;
004 | import java.util.List;
005 | import java.util.Optional;
006 |
007 | /** Contrato de persistência, independente de JDBC e de Servlets. */
008 | public interface ItemMidiaDao {
009 |     /** Insere um novo item e retorna a entidade com o identificador gerado. */
010 |     ItemMidia inserir(ItemMidia item);
011 |     /** Lista em ordem alfabética, ou pesquisa por título e autor/diretor. */
012 |     List<ItemMidia> buscar(String termo);
013 |     /** Retorna vazio quando o identificador não existe. */
014 |     Optional<ItemMidia> buscarPorId(long id);
015 |     /** Retorna false quando nenhuma linha foi alterada. */
016 |     boolean atualizar(ItemMidia item);
017 |     /** Retorna false quando o registro já não existe. */
018 |     boolean excluir(long id);
019 | }
```

src/main/java/br/edu/catalogo/dao/JdbcItemMidiaDao.java

```
001 | package br.edu.catalogo.dao;
002 |
003 | import br.edu.catalogo.config.FabricaConexao;
004 | import br.edu.catalogo.exception.PersistenciaException;
005 | import br.edu.catalogo.model.ItemMidia;
006 | import br.edu.catalogo.model.TipoMidia;
007 | import java.sql.PreparedStatement;
008 | import java.sql.ResultSet;
009 | import java.sql.SQLException;
010 | import java.sql.Statement;
011 | import java.util.ArrayList;
012 | import java.util.List;
013 | import java.util.Locale;
014 | import java.util.Optional;
015 |
016 | /** Único ponto de CRUD SQL: reutiliza conexão, mapeamento e parâmetros. */
017 | public final class JdbcItemMidiaDao implements ItemMidiaDao {
018 |     private static final String COLUNAS = "id, titulo, autor_diretor, ano_lancamento, genero,
 sinopse, tipo_midia";
019 |     private final FabricaConexao fabrica;
020 |     public JdbcItemMidiaDao(FabricaConexao fabrica) { this.fabrica = fabrica; }
021 |
022 |     @Override public ItemMidia inserir(ItemMidia item) {
023 |         String sql = "INSERT INTO item_midia (titulo, autor_diretor, ano_lancamento, genero,
sinopse, tipo_midia) VALUES (?, ?, ?, ?, ?, ?)";
024 |         try (var conexao = fabrica.abrir(); var comando = conexao.prepareStatement(sql,
Statement.RETURN_GENERATED_KEYS)) {
025 |             preencher(comando, item);
026 |             comando.executeUpdate();
027 |             try (var chaves = comando.getGeneratedKeys()) {
028 |                 if (!chaves.next()) throw new SQLException("Chave gerada não retornada.");
029 |                 return item.comId(chaves.getLong(1));
030 |             }
031 |         } catch (SQLException e) { throw new PersistenciaException("inserir item", e); }
032 |     }
033 |
034 |     @Override public List<ItemMidia> buscar(String termo) {
035 |         String sql = "SELECT " + COLUNAS + " FROM item_midia";
036 |         if (termo.isEmpty()) return consultar(sql + " ORDER BY LOWER(titulo), id", comando ->
 { });
037 |         // ! é o caractere de escape: %, _ e ! digitados são procurados literalmente.
038 |         String padrao = "%" + termo.toLowerCase(Locale.ROOT).replace("!", "!!").replace("%",
"!%").replace("_", "!_") + "%";
039 |         return consultar(sql + " WHERE LOWER(titulo) LIKE ? ESCAPE '!' OR
LOWER(autor_diretor) LIKE ? ESCAPE '!' ORDER BY LOWER(titulo), id", comando -> {
040 |             comando.setString(1, padrao);
041 |             comando.setString(2, padrao);
042 |         });
043 |     }
044 |
045 |     @Override public Optional<ItemMidia> buscarPorId(long id) {
046 |         return consultar("SELECT " + COLUNAS + " FROM item_midia WHERE id = ?", comando ->
comando.setLong(1, id))
047 |                 .stream().findFirst();
048 |     }
049 |
050 |     @Override public boolean atualizar(ItemMidia item) {
051 |         return modificar("UPDATE item_midia SET titulo=?, autor_diretor=?, ano_lancamento=?,
genero=?, sinopse=?, tipo_midia=? WHERE id=?", comando -> {
052 |             preencher(comando, item);
053 |             comando.setLong(7, item.getId());
054 |         });
055 |     }
056 |
057 |     @Override public boolean excluir(long id) {
058 |         return modificar("DELETE FROM item_midia WHERE id = ?", comando -> comando.setLong(1,
 id));
059 |     }
```

```
060 |
061 |     private List<ItemMidia> consultar(String sql, Parametros parametros) {
062 |         try (var conexao = fabrica.abrir(); var comando = conexao.prepareStatement(sql)) {
063 |             parametros.aplicar(comando);
064 |             try (var resultado = comando.executeQuery()) {
065 |                 List<ItemMidia> itens = new ArrayList<>();
066 |                 while (resultado.next()) itens.add(mapear(resultado));
067 |                 return itens;
068 |             }
069 |         } catch (SQLException e) { throw new PersistenciaException("consultar itens", e); }
070 |     }
071 |
072 |     private boolean modificar(String sql, Parametros parametros) {
073 |         try (var conexao = fabrica.abrir(); var comando = conexao.prepareStatement(sql)) {
074 |             parametros.aplicar(comando);
075 |             return comando.executeUpdate() > 0;
076 |         } catch (SQLException e) { throw new PersistenciaException("alterar item", e); }
077 |     }
078 |
079 |     private static void preencher(PreparedStatement comando, ItemMidia item) throws
SQLException {
080 |         comando.setString(1, item.getTitulo());
081 |         comando.setString(2, item.getAutorDiretor());
082 |         comando.setInt(3, item.getAnoLancamento());
083 |         comando.setString(4, item.getGenero());
084 |         comando.setString(5, item.getSinopse());
085 |         comando.setString(6, item.getTipoMidia().name());
086 |     }
087 |
088 |     private static ItemMidia mapear(ResultSet resultado) throws SQLException {
089 |         return new ItemMidia(resultado.getLong("id"), resultado.getString("titulo"),
090 |                 resultado.getString("autor_diretor"), resultado.getInt("ano_lancamento"),
091 |                 resultado.getString("genero"), resultado.getString("sinopse"),
092 |                 TipoMidia.valueOf(resultado.getString("tipo_midia")));
093 |     }
094 |
095 |     @FunctionalInterface private interface Parametros {
096 |         void aplicar(PreparedStatement comando) throws SQLException;
097 |     }
098 | }
```

src/main/java/br/edu/catalogo/exception/NaoEncontradoException.java

```
001 | package br.edu.catalogo.exception;
002 |
003 | /** Indica uma operação sobre um item que não existe mais no catálogo. */
004 | public final class NaoEncontradoException extends RuntimeException {
005 |     public NaoEncontradoException() { super("O item solicitado não foi encontrado."); }
006 | }
```

src/main/java/br/edu/catalogo/exception/PersistenciaException.java

```
001 | package br.edu.catalogo.exception;
002 |
003 | /** Isola SQLException da camada de negócio e conserva a causa para o log. */
004 | public final class PersistenciaException extends RuntimeException {
005 |     public PersistenciaException(String operacao, Throwable causa) {
006 |         super("Falha ao " + operacao + " no banco de dados.", causa);
007 |     }
008 | }
```

src/main/java/br/edu/catalogo/exception/ValidacaoException.java

```
001 | package br.edu.catalogo.exception;
002 |
003 | import java.util.Collections;
004 | import java.util.LinkedHashMap;
005 | import java.util.Map;
006 |
007 | /** Erros de negócio organizados por campo para a apresentação do formulário. */
008 | public final class ValidacaoException extends RuntimeException {
009 |     private final Map<String, String> erros;
010 |     public ValidacaoException(Map<String, String> erros) {
011 |         super("Revise os campos indicados.");
012 |         this.erros = Collections.unmodifiableMap(new LinkedHashMap<>(erros));
013 |     }
014 |     public Map<String, String> getErros() { return erros; }
015 | }
```

src/main/java/br/edu/catalogo/launcher/ServidorLocal.java

```
001 | package br.edu.catalogo.launcher;
002 |
003 | import br.edu.catalogo.config.FabricaConexao;
004 | import org.apache.catalina.startup.Tomcat;
005 | import org.apache.catalina.loader.WebappLoader;
006 | import java.nio.file.Files;
007 | import java.nio.file.Path;
008 |
009 | /** Inicializador de demonstração local; o WAR também funciona em Tomcat externo. */
010 | public final class ServidorLocal {
011 |     private ServidorLocal() { }
012 |     public static Tomcat iniciar(int porta, Path webapp, Path diretorio, FabricaConexao
fabrica) throws Exception {
013 |         Files.createDirectories(diretorio);
014 |         var tomcat = new Tomcat();
015 |         tomcat.setBaseDir(diretorio.toAbsolutePath().toString());
016 |         tomcat.setPort(porta);
017 |         tomcat.getConnector().setProperty("address", "127.0.0.1");
018 |         tomcat.getConnector().setMaxPostSize(64 * 1024);
019 |         var contexto = tomcat.addWebapp("", webapp.toAbsolutePath().toString());
020 |         contexto.setParentClassLoader(ServidorLocal.class.getClassLoader());
021 |         // No modo embutido, compartilha os tipos com o processo e seus testes.
022 |         var carregador = new WebappLoader();
023 |         carregador.setDelegate(true);
024 |         contexto.setLoader(carregador);
025 |         if (fabrica != null) contexto.getServletContext().setAttribute("fabricaConexao",
fabrica);
026 |         tomcat.start();
027 |         if (!contexto.getState().isAvailable()) {
028 |             tomcat.stop();
029 |             tomcat.destroy();
030 |             throw new IllegalStateException("A aplicação web não iniciou. Consulte o log do
Tomcat.");
031 |         }
032 |         return tomcat;
033 |     }
034 |     public static void main(String[] args) throws Exception {
035 |         int porta = Integer.parseInt(System.getenv().getOrDefault("CATALOGO_PORT", "8080"));
036 |         var tomcat = iniciar(porta, Path.of("target/catalogo"), Path.of(".runtime/tomcat"),
null);
037 |         Runtime.getRuntime().addShutdownHook(new Thread(() -> {
038 |             try { tomcat.stop(); tomcat.destroy(); }
039 |             catch (Exception e) { System.err.println("Não foi possível encerrar o servidor: "
 + e.getMessage()); }
040 |         }));
041 |         System.out.println("\nAcervo disponível em http://localhost:" +
tomcat.getConnector().getLocalPort() + "/itens\n");
042 |         tomcat.getServer().await();
043 |     }
044 | }
```

src/main/java/br/edu/catalogo/model/ItemMidia.java

```
001 | package br.edu.catalogo.model;
002 |
003 | import java.util.Objects;
004 |
005 | /** Entidade imutável: concentra o estado de uma obra, sem conhecer HTTP ou SQL. */
006 | public final class ItemMidia {
007 |     private final Long id;
008 |     private final String titulo;
009 |     private final String autorDiretor;
010 |     private final int anoLancamento;
011 |     private final String genero;
012 |     private final String sinopse;
013 |     private final TipoMidia tipoMidia;
014 |
015 |     public ItemMidia(Long id, String titulo, String autorDiretor, int anoLancamento,
016 |                      String genero, String sinopse, TipoMidia tipoMidia) {
017 |         this.id = id;
018 |         this.titulo = Objects.requireNonNull(titulo);
019 |         this.autorDiretor = Objects.requireNonNull(autorDiretor);
020 |         this.anoLancamento = anoLancamento;
021 |         this.genero = Objects.requireNonNull(genero);
022 |         this.sinopse = Objects.requireNonNull(sinopse);
023 |         this.tipoMidia = Objects.requireNonNull(tipoMidia);
024 |     }
025 |
026 |     /** Produz a versão persistida sem modificar o objeto original. */
027 |     public ItemMidia comId(long novoId) {
028 |         return new ItemMidia(novoId, titulo, autorDiretor, anoLancamento, genero, sinopse,
tipoMidia);
029 |     }
030 |     public Long getId() { return id; }
031 |     public String getTitulo() { return titulo; }
032 |     public String getAutorDiretor() { return autorDiretor; }
033 |     public int getAnoLancamento() { return anoLancamento; }
034 |     public String getGenero() { return genero; }
035 |     public String getSinopse() { return sinopse; }
036 |     public TipoMidia getTipoMidia() { return tipoMidia; }
037 | }
```

src/main/java/br/edu/catalogo/model/TipoMidia.java

```
001 | package br.edu.catalogo.model;
002 |
003 | /** Valores permitidos para o tipo de obra; o nome é persistido no banco. */
004 | public enum TipoMidia {
005 |     LIVRO("Livro"), FILME("Filme"), SERIE("Série");
006 |
007 |     private final String descricao;
008 |     TipoMidia(String descricao) { this.descricao = descricao; }
009 |     public String getDescricao() { return descricao; }
010 | }
```

src/main/java/br/edu/catalogo/service/ItemMidiaService.java

```
001 | package br.edu.catalogo.service;
002 |
003 | import br.edu.catalogo.dao.ItemMidiaDao;
004 | import br.edu.catalogo.exception.NaoEncontradoException;
005 | import br.edu.catalogo.exception.ValidacaoException;
006 | import br.edu.catalogo.model.ItemMidia;
007 | import br.edu.catalogo.model.TipoMidia;
008 | import java.time.Clock;
009 | import java.time.Year;
010 | import java.util.LinkedHashMap;
011 | import java.util.List;
012 | import java.util.Map;
013 | import java.util.Objects;
014 |
015 | /** Regras de negócio testáveis sem servidor HTTP e sem um banco real. */
016 | public final class ItemMidiaService {
017 |     private final ItemMidiaDao dao;
018 |     private final Clock relogio;
019 |     public ItemMidiaService(ItemMidiaDao dao) { this(dao, Clock.systemDefaultZone()); }
020 |     public ItemMidiaService(ItemMidiaDao dao, Clock relogio) {
021 |         this.dao = Objects.requireNonNull(dao);
022 |         this.relogio = Objects.requireNonNull(relogio);
023 |     }
024 |
025 |     /** Valida todos os campos antes de realizar qualquer escrita. */
026 |     public ItemMidia salvar(Long id, Map<String, String> campos) {
027 |         Map<String, String> erros = new LinkedHashMap<>();
028 |         String titulo = texto(campos, "titulo", "Título", 255, erros);
029 |         String autor = texto(campos, "autorDiretor", "Autor ou diretor", 255, erros);
030 |         String genero = texto(campos, "genero", "Gênero", 100, erros);
031 |         String sinopse = texto(campos, "sinopse", "Sinopse", 2000, erros);
032 |         int ano = 0;
033 |         int maximo = Year.now(relogio).getValue() + 1;
034 |         try {
035 |             ano = Integer.parseInt(limpar(campos.get("anoLancamento")));
036 |             if (ano < 1 || ano > maximo) erros.put("anoLancamento", "Informe um ano entre 1 e
 " + maximo + ".");
037 |         } catch (NumberFormatException e) { erros.put("anoLancamento", "Informe um ano
inteiro válido."); }
038 |         TipoMidia tipo = null;
039 |         try { tipo = TipoMidia.valueOf(limpar(campos.get("tipoMidia"))); }
040 |         catch (IllegalArgumentException e) { erros.put("tipoMidia", "Selecione Livro, Filme
ou Série."); }
041 |         if (!erros.isEmpty()) throw new ValidacaoException(erros);
042 |         var item = new ItemMidia(id, titulo, autor, ano, genero, sinopse, tipo);
043 |         if (id == null) return dao.inserir(item);
044 |         if (!dao.atualizar(item)) throw new NaoEncontradoException();
045 |         return item;
046 |     }
047 |
048 |     public List<ItemMidia> buscar(String termo) {
049 |         String normalizado = limpar(termo);
050 |         if (normalizado.length() > 255) throw new ValidacaoException(Map.of("q", "A busca
deve ter até 255 caracteres."));
051 |         return dao.buscar(normalizado);
052 |     }
053 |     public ItemMidia obter(long id) {
054 |         if (id <= 0) throw new NaoEncontradoException();
055 |         return dao.buscarPorId(id).orElseThrow(NaoEncontradoException::new);
056 |     }
057 |     public void excluir(long id) {
058 |         if (id <= 0 || !dao.excluir(id)) throw new NaoEncontradoException();
059 |     }
060 |     private static String texto(Map<String, String> campos, String chave, String nome, int
maximo, Map<String, String> erros) {
061 |         String valor = limpar(campos.get(chave));
062 |         if (valor.isEmpty()) erros.put(chave, nome + " é obrigatório.");
063 |         else if (valor.length() > maximo) erros.put(chave, nome + " deve ter até " + maximo +
```

```
 " caracteres.");
064 |         return valor;
065 |     }
066 |     private static String limpar(String valor) { return valor == null ? "" : valor.strip(); }
067 | }
```

src/main/java/br/edu/catalogo/web/AplicacaoListener.java

```
001 | package br.edu.catalogo.web;
002 |
003 | import br.edu.catalogo.config.BancoDados;
004 | import br.edu.catalogo.config.ConexaoJdbc;
005 | import br.edu.catalogo.config.FabricaConexao;
006 | import br.edu.catalogo.dao.JdbcItemMidiaDao;
007 | import br.edu.catalogo.service.ItemMidiaService;
008 | import jakarta.servlet.ServletContextEvent;
009 | import jakarta.servlet.ServletContextListener;
010 |
011 | /** Composition root: constrói o grafo de dependências uma única vez. */
012 | public final class AplicacaoListener implements ServletContextListener {
013 |     @Override public void contextInitialized(ServletContextEvent evento) {
014 |         var contexto = evento.getServletContext();
015 |         var fabrica = (FabricaConexao) contexto.getAttribute("fabricaConexao");
016 |         if (fabrica == null) fabrica = ConexaoJdbc.doAmbiente();
017 |         BancoDados.inicializar(fabrica);
018 |         contexto.setAttribute("itemService", new ItemMidiaService(new
JdbcItemMidiaDao(fabrica)));
019 |     }
020 | }
```

src/main/java/br/edu/catalogo/web/ItemMidiaServlet.java

```
001 | package br.edu.catalogo.web;
002 |
003 | import br.edu.catalogo.exception.NaoEncontradoException;
004 | import br.edu.catalogo.exception.PersistenciaException;
005 | import br.edu.catalogo.exception.ValidacaoException;
006 | import br.edu.catalogo.model.ItemMidia;
007 | import br.edu.catalogo.model.TipoMidia;
008 | import br.edu.catalogo.service.ItemMidiaService;
009 | import jakarta.servlet.ServletException;
010 | import jakarta.servlet.http.HttpServlet;
011 | import jakarta.servlet.http.HttpServletRequest;
012 | import jakarta.servlet.http.HttpServletResponse;
013 | import java.io.IOException;
014 | import java.time.Year;
015 | import java.util.LinkedHashMap;
016 | import java.util.Map;
017 | import java.util.logging.Level;
018 | import java.util.logging.Logger;
019 |
020 | /** Controlador MVC: traduz HTTP, delega ao serviço e escolhe a JSP/resposta. */
021 | public final class ItemMidiaServlet extends HttpServlet {
022 |     private static final Logger LOG = Logger.getLogger(ItemMidiaServlet.class.getName());
023 |     private ItemMidiaService servico;
024 |
025 |     @Override public void init() throws ServletException {
026 |         servico = (ItemMidiaService) getServletContext().getAttribute("itemService");
027 |         if (servico == null) throw new ServletException("Serviço não inicializado.");
028 |     }
029 |
030 |     @Override protected void doGet(HttpServletRequest req, HttpServletResponse res) throws
ServletException, IOException {
031 |         executar(req, res, () -> {
032 |             Object flash = req.getSession().getAttribute("flash");
033 |             req.getSession().removeAttribute("flash");
034 |             req.setAttribute("flash", flash);
035 |             switch (rota(req)) {
036 |                 case "/" -> {
037 |                     String termo = req.getParameter("q");
038 |                     req.setAttribute("termo", termo == null ? "" : termo);
039 |                     req.setAttribute("itens", servico.buscar(termo));
040 |                     exibir(req, res, "lista");
041 |                 }
042 |                 case "/novo" -> { prepararFormulario(req, null); exibir(req, res,
"formulario"); }
043 |                 case "/editar" -> { prepararFormulario(req, servico.obter(id(req)));
exibir(req, res, "formulario"); }
044 |                 case "/detalhes" -> { req.setAttribute("item", servico.obter(id(req)));
exibir(req, res, "detalhes"); }
045 |                 case "/salvar", "/excluir" -> { res.setHeader("Allow", "POST"); erro(req,
res, 405, "Esta operação exige o envio de um formulário."); }
046 |                 default -> erro(req, res, 404, "Página não encontrada.");
047 |             }
048 |         });
049 |     }
050 |
051 |     @Override protected void doPost(HttpServletRequest req, HttpServletResponse res) throws
ServletException, IOException {
052 |         executar(req, res, () -> {
053 |             switch (rota(req)) {
054 |                 case "/salvar" -> {
055 |                     Long identificador = req.getParameter("id") == null ||
req.getParameter("id").isBlank() ? null : id(req);
056 |                     Map<String, String> campos = lerCampos(req);
057 |                     try {
058 |                         ItemMidia item = servico.salvar(identificador, campos);
059 |                         req.getSession().setAttribute("flash", identificador == null ? "Item
cadastrado com sucesso." : "Alterações salvas com sucesso.");
060 |                         redirecionar(req, res, "/itens/detalhes?id=" + item.getId());
```

```
061 |                     } catch (ValidacaoException e) {
062 |                         res.setStatus(422);
063 |                         req.setAttribute("form", campos);
064 |                         req.setAttribute("id", identificador);
065 |                         req.setAttribute("erros", e.getErros());
066 |                         opcoes(req);
067 |                         exibir(req, res, "formulario");
068 |                     }
069 |                 }
070 |                 case "/excluir" -> {
071 |                     servico.excluir(id(req));
072 |                     req.getSession().setAttribute("flash", "Item excluído com sucesso.");
073 |                     redirecionar(req, res, "/itens");
074 |                 }
075 |                 default -> erro(req, res, 404, "Operação não encontrada.");
076 |             }
077 |         });
078 |     }
079 |
080 |     private void executar(HttpServletRequest req, HttpServletResponse res, Acao acao) throws
ServletException, IOException {
081 |         try { acao.executar(); }
082 |         catch (NaoEncontradoException e) { erro(req, res, 404, e.getMessage()); }
083 |         catch (NumberFormatException e) { erro(req, res, 400, "O identificador informado é
inválido."); }
084 |         catch (ValidacaoException e) { erro(req, res, 400,
e.getErros().values().iterator().next()); }
085 |         catch (PersistenciaException e) {
086 |             LOG.log(Level.SEVERE, "Falha de persistência ao atender a requisição", e);
087 |             erro(req, res, 503, "O catálogo está temporariamente indisponível. Tente
novamente em instantes.");
088 |         }
089 |     }
090 |
091 |     private static long id(HttpServletRequest req) {
092 |         String valor = req.getParameter("id");
093 |         if (valor == null || !valor.matches("[1-9][0-9]*")) throw new
NumberFormatException("ID inválido");
094 |         return Long.parseLong(valor);
095 |     }
096 |     private static String rota(HttpServletRequest req) {
097 |         return req.getPathInfo() == null || req.getPathInfo().isEmpty() ? "/" :
req.getPathInfo();
098 |     }
099 |     private static Map<String, String> lerCampos(HttpServletRequest req) {
100 |         Map<String, String> campos = new LinkedHashMap<>();
101 |         for (String campo : new String[]{"titulo", "autorDiretor", "anoLancamento", "genero",
 "sinopse", "tipoMidia"}) {
102 |             String valor = req.getParameter(campo);
103 |             campos.put(campo, valor == null ? "" : valor);
104 |         }
105 |         return campos;
106 |     }
107 |     private static void prepararFormulario(HttpServletRequest req, ItemMidia item) {
108 |         req.setAttribute("id", item == null ? null : item.getId());
109 |         req.setAttribute("form", item == null ? Map.of("tipoMidia", "LIVRO") : Map.of(
110 |                 "titulo", item.getTitulo(), "autorDiretor", item.getAutorDiretor(),
111 |                 "anoLancamento", String.valueOf(item.getAnoLancamento()), "genero",
item.getGenero(),
112 |                 "sinopse", item.getSinopse(), "tipoMidia", item.getTipoMidia().name()));
113 |         opcoes(req);
114 |     }
115 |     private static void opcoes(HttpServletRequest req) {
116 |         req.setAttribute("tipos", TipoMidia.values());
117 |         req.setAttribute("anoMaximo", Year.now().getValue() + 1);
118 |     }
119 |     private static void exibir(HttpServletRequest req, HttpServletResponse res, String
pagina) throws ServletException, IOException {
120 |         req.getRequestDispatcher("/WEB-INF/views/" + pagina + ".jsp").forward(req, res);
121 |     }
122 |     private static void erro(HttpServletRequest req, HttpServletResponse res, int status,
```

```
String mensagem) throws ServletException, IOException {
123 |         res.setStatus(status);
124 |         req.setAttribute("mensagemErro", mensagem);
125 |         exibir(req, res, "erro");
126 |     }
127 |     private static void redirecionar(HttpServletRequest req, HttpServletResponse res, String
caminho) {
128 |         // Post/Redirect/Get: atualizar o navegador não repete a gravação.
129 |         res.setStatus(303);
130 |         res.setHeader("Location", req.getContextPath() + caminho);
131 |     }
132 |     @FunctionalInterface private interface Acao {
133 |         void executar() throws ServletException, IOException;
134 |     }
135 | }
```

src/main/java/br/edu/catalogo/web/SegurancaFilter.java

```
001 | package br.edu.catalogo.web;
002 |
003 | import jakarta.servlet.Filter;
004 | import jakarta.servlet.FilterChain;
005 | import jakarta.servlet.ServletException;
006 | import jakarta.servlet.ServletRequest;
007 | import jakarta.servlet.ServletResponse;
008 | import jakarta.servlet.http.HttpServletRequest;
009 | import jakarta.servlet.http.HttpServletResponse;
010 | import java.io.IOException;
011 | import java.nio.charset.StandardCharsets;
012 | import java.security.MessageDigest;
013 | import java.security.SecureRandom;
014 | import java.util.HexFormat;
015 |
016 | /** Aplica UTF-8, cabeçalhos e proteção CSRF antes de chegar ao controlador. */
017 | public final class SegurancaFilter implements Filter {
018 |     private static final SecureRandom ALEATORIO = new SecureRandom();
019 |     @Override public void doFilter(ServletRequest request, ServletResponse response,
FilterChain chain)
020 |             throws IOException, ServletException {
021 |         var req = (HttpServletRequest) request;
022 |         var res = (HttpServletResponse) response;
023 |         req.setCharacterEncoding("UTF-8");
024 |         res.setCharacterEncoding("UTF-8");
025 |         res.setHeader("X-Content-Type-Options", "nosniff");
026 |         res.setHeader("X-Frame-Options", "DENY");
027 |         res.setHeader("Referrer-Policy", "same-origin");
028 |         res.setHeader("Content-Security-Policy", "default-src 'self'; style-src 'self';
script-src 'self'; img-src 'self' data:; form-action 'self'; frame-ancestors 'none'; base-uri
'self'");
029 |         res.setHeader("Cache-Control", "no-store");
030 |         var sessao = req.getSession();
031 |         String token;
032 |         synchronized (sessao) {
033 |             token = (String) sessao.getAttribute("csrfToken");
034 |             if (token == null) {
035 |                 byte[] bytes = new byte[32];
036 |                 ALEATORIO.nextBytes(bytes);
037 |                 token = HexFormat.of().formatHex(bytes);
038 |                 sessao.setAttribute("csrfToken", token);
039 |             }
040 |         }
041 |         if ("POST".equals(req.getMethod())) {
042 |             String recebido = req.getParameter("csrfToken");
043 |             if (recebido == null ||
!MessageDigest.isEqual(token.getBytes(StandardCharsets.UTF_8),
recebido.getBytes(StandardCharsets.UTF_8))) {
044 |                 res.setStatus(403);
045 |                 req.setAttribute("mensagemErro", "O formulário expirou ou não é válido.
Reabra a página e tente novamente.");
046 |                 req.getRequestDispatcher("/WEB-INF/views/erro.jsp").forward(req, res);
047 |                 return;
048 |             }
049 |         }
050 |         chain.doFilter(req, res);
051 |     }
052 | }
```

src/test/java/br/edu/catalogo/dao/JdbcItemMidiaDaoTest.java

```
001 | package br.edu.catalogo.dao;
002 |
003 | import br.edu.catalogo.config.BancoDados;
004 | import br.edu.catalogo.config.ConexaoJdbc;
005 | import br.edu.catalogo.exception.PersistenciaException;
006 | import br.edu.catalogo.model.ItemMidia;
007 | import br.edu.catalogo.model.TipoMidia;
008 | import java.nio.file.Path;
009 | import java.sql.SQLException;
010 | import java.util.UUID;
011 | import org.junit.jupiter.api.BeforeEach;
012 | import org.junit.jupiter.api.Test;
013 | import org.junit.jupiter.api.io.TempDir;
014 | import static org.junit.jupiter.api.Assertions.*;
015 |
016 | class JdbcItemMidiaDaoTest {
017 |     private JdbcItemMidiaDao dao;
018 |     private ConexaoJdbc fabrica;
019 |     @BeforeEach void preparar() {
020 |         fabrica = new ConexaoJdbc("jdbc:h2:mem:" + UUID.randomUUID() + ";DB_CLOSE_DELAY=-1",
"sa", "");
021 |         BancoDados.inicializar(fabrica);
022 |         dao = new JdbcItemMidiaDao(fabrica);
023 |     }
024 |     private ItemMidia item(String titulo, String autor) {
025 |         return new ItemMidia(null, titulo, autor, 1899, "Romance", "Sinopse da obra.",
TipoMidia.LIVRO);
026 |     }
027 |     @Test void crudRealComChaveGerada() {
028 |         var salvo = dao.inserir(item("Dom Casmurro", "Machado de Assis"));
029 |         assertTrue(salvo.getId() > 0);
030 |         assertEquals("Dom Casmurro",
dao.buscarPorId(salvo.getId()).orElseThrow().getTitulo());
031 |         var editado = new ItemMidia(salvo.getId(), "Título editado", "Outro autor", 2001,
"Drama", "Nova sinopse", TipoMidia.FILME);
032 |         assertTrue(dao.atualizar(editado));
033 |         var lido = dao.buscarPorId(salvo.getId()).orElseThrow();
034 |         assertAll(() -> assertEquals("Título editado", lido.getTitulo()), () ->
assertEquals("Outro autor", lido.getAutorDiretor()),
035 |                 () -> assertEquals(2001, lido.getAnoLancamento()), () ->
assertEquals("Drama", lido.getGenero()),
036 |                 () -> assertEquals("Nova sinopse", lido.getSinopse()), () ->
assertEquals(TipoMidia.FILME, lido.getTipoMidia()));
037 |         assertTrue(dao.excluir(salvo.getId()));
038 |         assertFalse(dao.excluir(salvo.getId()));
039 |         assertTrue(dao.buscarPorId(salvo.getId()).isEmpty());
040 |     }
041 |     @Test void buscaPorTituloOuAutorSemDiferenciarMaiusculas() {
042 |         dao.inserir(item("Dom Casmurro", "Machado de Assis"));
043 |         dao.inserir(item("Outra obra", "Outro autor"));
044 |         assertEquals(1, dao.buscar("CASMURRO").size());
045 |         assertEquals(1, dao.buscar("mAcHaDo").size());
046 |         assertEquals(2, dao.buscar("").size());
047 |     }
048 |     @Test void aspasEInjecaoSaoDadosLiterais() {
049 |         dao.inserir(item("O livro de D'Ávila", "Autor"));
050 |         assertEquals(1, dao.buscar("D'Ávila").size());
051 |         assertEquals(0, dao.buscar("' OR 1=1 --").size());
052 |         assertEquals(1, dao.buscar("").size());
053 |     }
054 |     @Test void curingasNaoAmpliamBuscaLiteral() {
055 |         dao.inserir(item("100% real_!", "Autor"));
056 |         dao.inserir(item("Outro título", "Outro autor"));
057 |         assertEquals(1, dao.buscar("%").size());
058 |         assertEquals(1, dao.buscar("_").size());
059 |         assertEquals(1, dao.buscar("!").size());
060 |     }
061 |     @Test void falhaSqlPreservaCausaSemExporJdbcAoServico() {
```

```
062 |         var indisponivel = new JdbcItemMidiaDao(() -> { throw new SQLException("Falha
simulada"); });
063 |         assertInstanceOf(SQLException.class, assertThrows(PersistenciaException.class, () ->
indisponivel.buscar("")).getCause());
064 |     }
065 |     @Test void restricoesDoBancoProtegemIntegridade() {
066 |         assertThrows(PersistenciaException.class, () -> dao.inserir(new ItemMidia(null, "",
"Autor", 2000, "Drama", "Sinopse", TipoMidia.LIVRO)));
067 |         assertThrows(PersistenciaException.class, () -> dao.inserir(new ItemMidia(null,
"Título", "Autor", 0, "Drama", "Sinopse", TipoMidia.LIVRO)));
068 |     }
069 |     @Test void reinicializarEstruturaNaoApagaDados() {
070 |         dao.inserir(item("Preservado", "Autor"));
071 |         BancoDados.inicializar(fabrica);
072 |         assertEquals(1, dao.buscar("").size());
073 |     }
074 |     @Test void dadosSobrevivemAoFechamentoEReabertura(@TempDir Path pasta) {
075 |         String url = "jdbc:h2:file:" +
pasta.resolve("catalogo").toAbsolutePath().toString().replace('\\', '/');
076 |         var arquivo = new ConexaoJdbc(url, "sa", "");
077 |         BancoDados.inicializar(arquivo);
078 |         var primeiro = new JdbcItemMidiaDao(arquivo);
079 |         var salvo = primeiro.inserir(item("Persistido em disco", "Autor"));
080 |         // Não há DB_CLOSE_DELAY: o H2 fecha o arquivo ao fechar a última conexão.
081 |         var reaberto = new JdbcItemMidiaDao(new ConexaoJdbc(url, "sa", ""));
082 |         assertEquals("Persistido em disco",
reaberto.buscarPorId(salvo.getId()).orElseThrow().getTitulo());
083 |     }
084 | }
```

src/test/java/br/edu/catalogo/service/ItemMidiaServiceTest.java

```
001 | package br.edu.catalogo.service;
002 |
003 | import br.edu.catalogo.exception.NaoEncontradoException;
004 | import br.edu.catalogo.exception.ValidacaoException;
005 | import br.edu.catalogo.support.ItemMidiaDaoEmMemoria;
006 | import java.time.Clock;
007 | import java.time.Instant;
008 | import java.time.ZoneOffset;
009 | import java.util.LinkedHashMap;
010 | import java.util.Map;
011 | import org.junit.jupiter.api.BeforeEach;
012 | import org.junit.jupiter.api.Test;
013 | import org.junit.jupiter.params.ParameterizedTest;
014 | import org.junit.jupiter.params.provider.CsvSource;
015 | import org.junit.jupiter.params.provider.ValueSource;
016 | import static org.junit.jupiter.api.Assertions.*;
017 |
018 | class ItemMidiaServiceTest {
019 |     private ItemMidiaDaoEmMemoria dao;
020 |     private ItemMidiaService servico;
021 |     @BeforeEach void preparar() {
022 |         dao = new ItemMidiaDaoEmMemoria();
023 |         servico = new ItemMidiaService(dao,
Clock.fixed(Instant.parse("2026-01-01T00:00:00Z"), ZoneOffset.UTC));
024 |     }
025 |     private Map<String, String> valido() {
026 |         return new LinkedHashMap<>(Map.of("titulo", "  Dom Casmurro  ", "autorDiretor", "
Machado de Assis ",
027 |                 "anoLancamento", "1899", "genero", "Romance", "sinopse", "Um narrador
revisita sua juventude.", "tipoMidia", "LIVRO"));
028 |     }
029 |     @Test void cadastraENormalizaSemHttpOuJdbc() {
030 |         var item = servico.salvar(null, valido());
031 |         assertNotNull(item.getId());
032 |         assertEquals("Dom Casmurro", item.getTitulo());
033 |         assertEquals("Machado de Assis", item.getAutorDiretor());
034 |         assertEquals(1, dao.escritas);
035 |     }
036 |     @Test void acumulaTodosOsErrosSemEscreverNoDao() {
037 |         var erro = assertThrows(ValidacaoException.class, () -> servico.salvar(null,
Map.of()));
038 |         assertEquals(6, erro.getErros().size());
039 |         assertEquals(0, dao.escritas);
040 |         assertThrows(UnsupportedOperationException.class, () -> erro.getErros().put("x",
"y"));
041 |     }
042 |     @ParameterizedTest @CsvSource({"titulo,256", "autorDiretor,256", "genero,101",
"sinopse,2001"})
043 |     void rejeitaComprimentosAcimaDoLimite(String campo, int tamanho) {
044 |         var dados = valido(); dados.put(campo, "x".repeat(tamanho));
045 |         assertTrue(assertThrows(ValidacaoException.class, () -> servico.salvar(null,
dados)).getErros().containsKey(campo));
046 |         assertEquals(0, dao.escritas);
047 |     }
048 |     @ParameterizedTest @CsvSource({"titulo,255", "autorDiretor,255", "genero,100",
"sinopse,2000"})
049 |     void aceitaComprimentosNoLimite(String campo, int tamanho) {
050 |         var dados = valido(); dados.put(campo, "x".repeat(tamanho));
051 |         assertDoesNotThrow(() -> servico.salvar(null, dados));
052 |     }
053 |     @ParameterizedTest @ValueSource(strings={"texto", "0", "-1", "2028",
"9999999999999999"})
054 |     void rejeitaAnoInvalido(String ano) {
055 |         var dados = valido(); dados.put("anoLancamento", ano);
056 |         assertTrue(assertThrows(ValidacaoException.class, () -> servico.salvar(null,
dados)).getErros().containsKey("anoLancamento"));
```

```
057 |         assertEquals(0, dao.escritas);
058 |     }
059 |     @ParameterizedTest @ValueSource(strings={"1", "2027"})
060 |     void aceitaLimitesDoAno(String ano) {
061 |         var dados = valido(); dados.put("anoLancamento", ano);
062 |         assertDoesNotThrow(() -> servico.salvar(null, dados));
063 |     }
064 |     @Test void rejeitaTipoNaoPrevisto() {
065 |         var dados = valido(); dados.put("tipoMidia", "JOGO");
066 |         assertTrue(assertThrows(ValidacaoException.class, () -> servico.salvar(null,
dados)).getErros().containsKey("tipoMidia"));
067 |     }
068 |     @Test void atualizaMantendoIdentificador() {
069 |         var item = servico.salvar(null, valido());
070 |         var dados = valido(); dados.put("titulo", "Novo título");
071 |         assertEquals(item.getId(), servico.salvar(item.getId(), dados).getId());
072 |         assertEquals("Novo título", servico.obter(item.getId()).getTitulo());
073 |         assertEquals(1, dao.itens.size());
074 |     }
075 |     @Test void naoTransformaEdicaoInexistenteEmCadastro() {
076 |         assertThrows(NaoEncontradoException.class, () -> servico.salvar(999L, valido()));
077 |         assertEquals(0, dao.escritas);
078 |     }
079 |     @Test void consultaEExclusaoInexistentesSaoExplicitas() {
080 |         assertThrows(NaoEncontradoException.class, () -> servico.obter(999));
081 |         assertThrows(NaoEncontradoException.class, () -> servico.obter(0));
082 |         assertThrows(NaoEncontradoException.class, () -> servico.excluir(999));
083 |     }
084 |     @Test void normalizaTermoEAceitaBuscaVazia() {
085 |         servico.buscar("  Machado  "); assertEquals("Machado", dao.ultimoTermo);
086 |         servico.buscar(null); assertEquals("", dao.ultimoTermo);
087 |         assertThrows(ValidacaoException.class, () -> servico.buscar("x".repeat(256)));
088 |     }
089 |     @Test void excluiItemExistente() {
090 |         var item = servico.salvar(null, valido());
091 |         servico.excluir(item.getId());
092 |         assertTrue(dao.itens.isEmpty());
093 |     }
094 | }
```

src/test/java/br/edu/catalogo/support/ItemMidiaDaoEmMemoria.java

```
001 | package br.edu.catalogo.support;
002 |
003 | import br.edu.catalogo.dao.ItemMidiaDao;
004 | import br.edu.catalogo.model.ItemMidia;
005 | import java.util.ArrayList;
006 | import java.util.LinkedHashMap;
007 | import java.util.List;
008 | import java.util.Map;
009 | import java.util.Optional;
010 |
011 | /** Dublê exclusivo dos testes: demonstra o polimorfismo do contrato DAO. */
012 | public final class ItemMidiaDaoEmMemoria implements ItemMidiaDao {
013 |     public final Map<Long, ItemMidia> itens = new LinkedHashMap<>();
014 |     public int escritas;
015 |     public String ultimoTermo;
016 |     private long sequencia;
017 |     @Override public ItemMidia inserir(ItemMidia item) {
018 |         escritas++;
019 |         ItemMidia salvo = item.comId(++sequencia);
020 |         itens.put(salvo.getId(), salvo);
021 |         return salvo;
022 |     }
023 |     @Override public List<ItemMidia> buscar(String termo) { ultimoTermo = termo; return new
ArrayList<>(itens.values()); }
024 |     @Override public Optional<ItemMidia> buscarPorId(long id) { return
Optional.ofNullable(itens.get(id)); }
025 |     @Override public boolean atualizar(ItemMidia item) {
026 |         if (!itens.containsKey(item.getId())) return false;
027 |         escritas++;
028 |         itens.put(item.getId(), item);
029 |         return true;
030 |     }
031 |     @Override public boolean excluir(long id) { return itens.remove(id) != null; }
032 | }
```

src/test/java/br/edu/catalogo/web/CatalogoHttpIT.java

```
001 | package br.edu.catalogo.web;
002 |
003 | import br.edu.catalogo.config.ConexaoJdbc;
004 | import br.edu.catalogo.dao.JdbcItemMidiaDao;
005 | import br.edu.catalogo.launcher.ServidorLocal;
006 | import br.edu.catalogo.model.ItemMidia;
007 | import br.edu.catalogo.model.TipoMidia;
008 | import java.net.CookieManager;
009 | import java.net.URI;
010 | import java.net.URLEncoder;
011 | import java.net.http.HttpClient;
012 | import java.net.http.HttpRequest;
013 | import java.net.http.HttpResponse;
014 | import java.nio.charset.StandardCharsets;
015 | import java.nio.file.Path;
016 | import java.sql.SQLException;
017 | import java.util.LinkedHashMap;
018 | import java.util.Map;
019 | import java.util.UUID;
020 | import java.util.concurrent.atomic.AtomicBoolean;
021 | import java.util.regex.Pattern;
022 | import java.util.stream.Collectors;
023 | import org.apache.catalina.startup.Tomcat;
024 | import org.junit.jupiter.api.*;
025 | import org.junit.jupiter.api.io.TempDir;
026 | import static org.junit.jupiter.api.Assertions.*;
027 |
028 | /** Testa HTTP -> filtro -> Servlet -> serviço -> JDBC -> JSP no Tomcat real. */
029 | @TestInstance(TestInstance.Lifecycle.PER_CLASS)
030 | class CatalogoHttpIT {
031 |     @TempDir static Path temporario;
032 |     private Tomcat tomcat;
033 |     private ConexaoJdbc fabrica;
034 |     private JdbcItemMidiaDao dao;
035 |     private HttpClient cliente;
036 |     private String base;
037 |     private final AtomicBoolean falha = new AtomicBoolean();
038 |
039 |     @BeforeAll void iniciar() throws Exception {
040 |         fabrica = new ConexaoJdbc("jdbc:h2:mem:http-" + UUID.randomUUID() +
";DB_CLOSE_DELAY=-1", "sa", "");
041 |         tomcat = ServidorLocal.iniciar(0, Path.of("target/catalogo"), temporario, () -> {
042 |             if (falha.get()) throw new SQLException("credencial-interna-que-nao-pode-vazar");
043 |             return fabrica.abrir();
044 |         });
045 |         base = "http://127.0.0.1:" + tomcat.getConnector().getLocalPort();
046 |         dao = new JdbcItemMidiaDao(fabrica);
047 |     }
048 |     @BeforeEach void limpar() throws Exception {
049 |         cliente = HttpClient.newBuilder().cookieHandler(new CookieManager()).build();
050 |         try (var conexao = fabrica.abrir(); var comando = conexao.createStatement()) {
comando.executeUpdate("DELETE FROM item_midia"); }
051 |     }
052 |     @AfterAll void encerrar() throws Exception { if (tomcat != null) { tomcat.stop();
tomcat.destroy(); } }
053 |     private HttpResponse<String> get(String rota) throws Exception {
054 |         return cliente.send(HttpRequest.newBuilder(URI.create(base + rota)).GET().build(),
HttpResponse.BodyHandlers.ofString());
055 |     }
056 |     private HttpResponse<String> post(String rota, Map<String, String> campos) throws
Exception {
057 |         String corpo = campos.entrySet().stream().map(e -> codificar(e.getKey()) + "=" +
codificar(e.getValue())).collect(Collectors.joining("&"));
058 |         return cliente.send(HttpRequest.newBuilder(URI.create(base + rota)).header("Content-
Type", "application/x-www-form-urlencoded; charset=UTF-8")
059 |                 .POST(HttpRequest.BodyPublishers.ofString(corpo)).build(),
HttpResponse.BodyHandlers.ofString());
060 |     }
```

```
061 |     private static String codificar(String valor) { return URLEncoder.encode(valor,
StandardCharsets.UTF_8); }
062 |     private Map<String, String> formulario() throws Exception {
063 |         var pagina = get("/itens/novo");
064 |         assertEquals(200, pagina.statusCode(), pagina.body());
065 |         var matcher = Pattern.compile("name=\"csrfToken\"
value=\"([^\"]+)\"").matcher(pagina.body());
066 |         assertTrue(matcher.find());
067 |         return new LinkedHashMap<>(Map.of("csrfToken", matcher.group(1), "titulo", "Dom
Casmurro", "autorDiretor", "Machado de Assis",
068 |                 "anoLancamento", "1899", "genero", "Romance", "sinopse", "Um narrador
revisita a juventude.", "tipoMidia", "LIVRO"));
069 |     }
070 |     private ItemMidia cadastrarDireto(String titulo) {
071 |         return dao.inserir(new ItemMidia(null, titulo, "Machado de Assis", 1899, "Romance",
"Sinopse de teste.", TipoMidia.LIVRO));
072 |     }
073 |     @Test void fluxoCompletoCadastroBuscaEdicaoExclusao() throws Exception {
074 |         var campos = formulario();
075 |         var cadastro = post("/itens/salvar", campos);
076 |         assertEquals(303, cadastro.statusCode(), cadastro.body());
077 |         String detalhes = cadastro.headers().firstValue("Location").orElseThrow();
078 |         assertTrue(get(detalhes).body().contains("Dom Casmurro"));
079 |         assertTrue(get("/itens?q=machado").body().contains("Dom Casmurro"));
080 |         var item = dao.buscar("").get(0);
081 |         assertTrue(get("/itens/editar?id=" + item.getId()).body().contains("Dom Casmurro"));
082 |         campos.put("id", item.getId().toString()); campos.put("titulo", "Dom Casmurro —
edição revisada");
083 |         assertEquals(303, post("/itens/salvar", campos).statusCode());
084 |         assertTrue(get(detalhes).body().contains("edição revisada"));
085 |         assertEquals(1, dao.buscar("").size());
086 |         assertEquals(303, post("/itens/excluir", Map.of("id", item.getId().toString(),
"csrfToken", campos.get("csrfToken"))).statusCode());
087 |         assertTrue(dao.buscar("").isEmpty());
088 |         assertEquals(404, get(detalhes).statusCode());
089 |     }
090 |     @Test void erroDeValidacaoPreservaDadosNaoGravaEPermiteCorrigir() throws Exception {
091 |         var campos = formulario(); campos.put("anoLancamento", "abc");
092 |         var resposta = post("/itens/salvar", campos);
093 |         assertEquals(422, resposta.statusCode());
094 |         assertTrue(resposta.body().contains("Dom Casmurro"));
095 |         assertTrue(resposta.body().contains("ano inteiro válido"));
096 |         assertTrue(dao.buscar("").isEmpty());
097 |         campos.put("anoLancamento", "1899");
098 |         assertEquals(303, post("/itens/salvar", campos).statusCode());
099 |     }
100 |     @Test void rejeitaCsrfAusenteEIncorreto() throws Exception {
101 |         var campos = formulario(); campos.remove("csrfToken");
102 |         assertEquals(403, post("/itens/salvar", campos).statusCode());
103 |         campos.put("csrfToken", "invalido");
104 |         assertEquals(403, post("/itens/salvar", campos).statusCode());
105 |         assertTrue(dao.buscar("").isEmpty());
106 |     }
107 |     @Test void getNaoExcluiEIdentificadoresInvalidosNaoQuebramAplicacao() throws Exception {
108 |         var item = cadastrarDireto("Obra preservada");
109 |         assertEquals(405, get("/itens/excluir?id=" + item.getId()).statusCode());
110 |         assertTrue(dao.buscarPorId(item.getId()).isPresent());
111 |         assertEquals(400, get("/itens/detalhes?id=abc").statusCode());
112 |         assertEquals(400, get("/itens/detalhes?id=99999999999999999999").statusCode());
113 |         assertEquals(400, get("/itens/detalhes").statusCode());
114 |         assertEquals(404, get("/itens/detalhes?id=999999").statusCode());
115 |         assertEquals(404, get("/itens/rota-inexistente").statusCode());
116 |     }
117 |     @Test void escapaXssNaListaDetalhesEFormulario() throws Exception {
118 |         String carga = "<script>alert('x')</script>";
119 |         var item = cadastrarDireto(carga);
120 |         for (String rota : new String[]{"/itens", "/itens/detalhes?id=" + item.getId(),
"/itens/editar?id=" + item.getId()}) {
121 |             var pagina = get(rota);
122 |             assertEquals(200, pagina.statusCode());
123 |             assertFalse(pagina.body().contains(carga));
```

```
124 |             assertTrue(pagina.body().contains("&lt;script&gt;"), pagina.body());
125 |         }
126 |     }
127 |     @Test void buscaMaliciosaNaoRetornaTodosOsItens() throws Exception {
128 |         cadastrarDireto("Obra segura");
129 |         var pagina = get("/itens?q=" + codificar("' OR 1=1 --"));
130 |         assertEquals(200, pagina.statusCode());
131 |         assertFalse(pagina.body().contains("Obra segura"));
132 |         assertEquals(1, dao.buscar("").size());
133 |     }
134 |     @Test void cabecalhosEViewsInternas() throws Exception {
135 |         var pagina = get("/itens");
136 |         assertEquals(200, pagina.statusCode());
137 |         assertTrue(pagina.body().contains("Meu catálogo"));
138 |         assertTrue(pagina.body().contains("Histórias que ficam com você."));
139 |         assertEquals("nosniff", pagina.headers().firstValue("X-Content-Type-
Options").orElseThrow());
140 |         assertTrue(pagina.headers().firstValue("Content-Security-
Policy").orElseThrow().contains("frame-ancestors 'none'"));
141 |         assertEquals(404, get("/WEB-INF/views/lista.jsp").statusCode());
142 |         assertTrue(get("/assets/style.css").body().contains(".card-grid"));
143 |         assertTrue(get("/assets/app.js").body().contains("sinopse"));
144 |     }
145 |     @Test void indisponibilidadeDoBancoRetornaMensagemSemDetalhesInternos() throws Exception
{
146 |         falha.set(true);
147 |         try {
148 |             var pagina = get("/itens");
149 |             assertEquals(503, pagina.statusCode());
150 |             assertTrue(pagina.body().contains("temporariamente indisponível"));
151 |             assertFalse(pagina.body().contains("credencial-interna"));
152 |             assertFalse(pagina.body().contains("SQLException"));
153 |         } finally { falha.set(false); }
154 |     }
155 | }
```

pom.xml

```
001 | <?xml version="1.0" encoding="UTF-8"?>
002 | <project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
003 |   <modelVersion>4.0.0</modelVersion>
004 |   <groupId>br.edu.projeto</groupId>
005 |   <artifactId>catalogo</artifactId>
006 |   <version>1.0.0</version>
007 |   <packaging>war</packaging>
008 |   <name>Acervo — Catálogo de mídias</name>
009 |   <properties>
010 |     <maven.compiler.release>17</maven.compiler.release>
011 |     <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
012 |     <tomcat.version>10.1.61</tomcat.version>
013 |     <junit.version>5.12.2</junit.version>
014 |   </properties>
015 |   <dependencies>
016 |     <dependency><groupId>jakarta.servlet</groupId><artifactId>jakarta.servlet-
api</artifactId><version>6.0.0</version><scope>provided</scope></dependency>
017 |     <dependency><groupId>org.apache.tomcat.embed</groupId><artifactId>tomcat-embed-
core</artifactId><version>${tomcat.version}</version></dependency>
018 |     <dependency><groupId>org.apache.tomcat.embed</groupId><artifactId>tomcat-embed-
jasper</artifactId><version>${tomcat.version}</version></dependency>
019 |
<dependency><groupId>jakarta.servlet.jsp.jstl</groupId><artifactId>jakarta.servlet.jsp.jstl-
api</artifactId><version>3.0.2</version></dependency>
020 |     <dependency><groupId>org.glassfish.web</groupId><artifactId>jakarta.servlet.jsp.jstl</art
ifactId><version>3.0.1</version></dependency>
021 |     <dependency><groupId>com.h2database</groupId><artifactId>h2</artifactId><version>2.3.232<
/version></dependency>
022 |     <dependency><groupId>org.junit.jupiter</groupId><artifactId>junit-
jupiter</artifactId><version>${junit.version}</version><scope>test</scope></dependency>
023 |   </dependencies>
024 |   <build>
025 |     <finalName>catalogo</finalName>
026 |     <plugins>
027 |       <plugin><groupId>org.apache.maven.plugins</groupId><artifactId>maven-compiler-
plugin</artifactId><version>3.14.0</version></plugin>
028 |       <plugin><groupId>org.apache.maven.plugins</groupId><artifactId>maven-surefire-plugin</a
rtifactId><version>3.5.3</version><configuration><useModulePath>false</useModulePath></configuratio
n></plugin>
029 |       <plugin><groupId>org.apache.maven.plugins</groupId><artifactId>maven-failsafe-
plugin</artifactId><version>3.5.3</version><executions><execution><goals><goal>integration-
test</goal><goal>verify</goal></goals></execution></executions></plugin>
030 |       <plugin><groupId>org.apache.maven.plugins</groupId><artifactId>maven-war-
plugin</artifactId><version>3.4.0</version><configuration><packagingExcludes>WEB-INF/lib/tomcat-
embed-*.jar,WEB-INF/lib/ecj-*.jar</packagingExcludes></configuration></plugin>
031 |       <plugin><groupId>org.apache.maven.plugins</groupId><artifactId>maven-dependency-plugin<
/artifactId><version>3.8.1</version><executions><execution><id>runtime</id><phase>package</phase><g
oals><goal>copy-dependencies</goal></goals><configuration><includeScope>runtime</includeScope><outp
utDirectory>${project.build.directory}/dependency</outputDirectory></configuration></execution></ex
ecutions></plugin>
032 |     </plugins>
033 |   </build>
034 | </project>
```

src/main/resources/db/schema.sql

```
001 | CREATE TABLE IF NOT EXISTS item_midia (
002 |     id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
003 |     titulo VARCHAR(255) NOT NULL CHECK (CHAR_LENGTH(TRIM(titulo)) > 0),
004 |     autor_diretor VARCHAR(255) NOT NULL CHECK (CHAR_LENGTH(TRIM(autor_diretor)) > 0),
005 |     ano_lancamento INTEGER NOT NULL CHECK (ano_lancamento BETWEEN 1 AND 9999),
006 |     genero VARCHAR(100) NOT NULL CHECK (CHAR_LENGTH(TRIM(genero)) > 0),
007 |     sinopse VARCHAR(2000) NOT NULL CHECK (CHAR_LENGTH(TRIM(sinopse)) > 0),
008 |     tipo_midia VARCHAR(10) NOT NULL CHECK (tipo_midia IN ('LIVRO', 'FILME', 'SERIE'))
009 | );
```

src/main/webapp/WEB-INF/web.xml

```
001 | <?xml version="1.0" encoding="UTF-8"?>
002 | <web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee https://jakarta.ee/xml/ns/jakartaee/web-
app_6_0.xsd" version="6.0">
003 |   <display-name>Acervo</display-name>
004 |   <listener><listener-class>br.edu.catalogo.web.AplicacaoListener</listener-class></listener>
005 |   <filter><filter-name>seguranca</filter-name><filter-
class>br.edu.catalogo.web.SegurancaFilter</filter-class></filter>
006 |   <filter-mapping><filter-name>seguranca</filter-name><url-pattern>/*</url-pattern></filter-
mapping>
007 |   <servlet><servlet-name>itens</servlet-name><servlet-
class>br.edu.catalogo.web.ItemMidiaServlet</servlet-class><load-on-startup>1</load-on-
startup></servlet>
008 |   <servlet-mapping><servlet-name>itens</servlet-name><url-pattern>/itens/*</url-
pattern></servlet-mapping>
009 |   <session-config><session-timeout>30</session-timeout><cookie-config><http-only>true</http-
only></cookie-config><tracking-mode>COOKIE</tracking-mode></session-config>
010 |   <welcome-file-list><welcome-file>index.jsp</welcome-file></welcome-file-list>
011 |   <error-page><error-code>404</error-code><location>/WEB-
INF/views/erro.jsp</location></error-page>
012 |   <error-page><error-code>500</error-code><location>/WEB-
INF/views/erro.jsp</location></error-page>
013 | </web-app>
```

src/main/webapp/WEB-INF/views/cabecalho.jspf

```
001 | <%@ page pageEncoding="UTF-8" %>
002 | <!DOCTYPE html>
003 | <html lang="pt-BR">
004 | <head>
005 |   <meta charset="UTF-8">
006 |   <meta name="viewport" content="width=device-width, initial-scale=1">
007 |   <meta name="description" content="Seu catálogo pessoal de livros, filmes e séries.">
008 |   <title><c:out value="${tituloPagina}"/> · Acervo</title>
009 |   <link rel="stylesheet" href="${pageContext.request.contextPath}/assets/style.css">
010 |   <script defer src="${pageContext.request.contextPath}/assets/app.js"></script>
011 | </head>
012 | <body>
013 | <a class="skip-link" href="#conteudo">Pular para o conteúdo</a>
014 | <header class="site-header"><div class="container header-inner">
015 |   <a class="brand" href="${pageContext.request.contextPath}/itens"><span class="brand-icon"
aria-hidden="true">a.</span> acervo<span class="brand-dot">.</span></a>
016 |   <nav aria-label="Navegação principal"><a
href="${pageContext.request.contextPath}/itens">Meu catálogo</a><a class="button button-small"
href="${pageContext.request.contextPath}/itens/novo">+ Adicionar item</a></nav>
017 | </div></header>
018 | <main id="conteudo" class="container">
019 | <c:if test="${not empty flash}"><div class="notice success" role="status"><c:out
value="${flash}"/></div></c:if>
```

src/main/webapp/WEB-INF/views/detalhes.jsp

```
001 | <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
002 | <%@ include file="tags.jspf" %>
003 | <c:set var="tituloPagina" value="${item.titulo}"/>
004 | <%@ include file="cabecalho.jspf" %>
005 | <a class="back-link" href="${pageContext.request.contextPath}/itens">← Voltar ao catálogo</a>
006 | <article class="detail-layout"><div class="detail-art card-art art-${item.tipoMidia}"><span
class="art-label"><c:out value="${item.tipoMidia.descricao}"/></span><span class="art-letter" aria-
hidden="true"><c:out value="${fn:substring(item.titulo, 0, 1)}"/></span><span class="art-
year">${item.anoLancamento}</span></div><div class="detail-content"><p class="eyebrow"><c:out
value="${item.genero}"/></p><h1><c:out value="${item.titulo}"/></h1><p class="detail-author"><c:out
 value="${item.autorDiretor}"/></p><dl class="metadata"><div><dt>Tipo</dt><dd><c:out value="${item.
tipoMidia.descricao}"/></dd></div><div><dt>Ano</dt><dd>${item.anoLancamento}</dd></div><div><dt>Ide
ntificador</dt><dd>#${item.id}</dd></div></dl><h2>Sobre a obra</h2><p class="full-synopsis"><c:out
value="${item.sinopse}"/></p><div class="detail-actions"><a class="button"
href="${pageContext.request.contextPath}/itens/editar?id=${item.id}">Editar informações</a><button
type="button" class="button button-danger" data-open-delete>Excluir item</button></div>
007 | <details class="delete-panel" id="confirmar-exclusao"><summary>Confirmar
exclusão</summary><p>Excluir “<c:out value="${item.titulo}"/>” do catálogo? Esta ação não pode ser
desfeita.</p><form action="${pageContext.request.contextPath}/itens/excluir" method="post"><input
type="hidden" name="csrfToken" value="${sessionScope.csrfToken}"><input type="hidden" name="id"
value="${item.id}"><button type="submit" class="button button-danger">Sim, excluir este
item</button></form></details></div></article>
008 | <%@ include file="rodape.jspf" %>
```

src/main/webapp/WEB-INF/views/erro.jsp

```
001 | <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
002 | <%@ include file="tags.jspf" %>
003 | <c:set var="tituloPagina" value="Não foi possível concluir"/>
004 | <%@ include file="cabecalho.jspf" %>
005 | <section class="empty-state error-page"><p class="eyebrow">VAMOS TENTAR NOVAMENTE</p><h1>Algo
 saiu do roteiro.</h1><p><c:out value="${empty mensagemErro ? 'Não foi possível abrir esta página.
Volte ao catálogo para continuar.' : mensagemErro}"/></p><a class="button"
href="${pageContext.request.contextPath}/itens">Voltar ao catálogo</a></section>
006 | <%@ include file="rodape.jspf" %>
```

src/main/webapp/WEB-INF/views/formulario.jsp

```
001 | <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
002 | <%@ include file="tags.jspf" %>
003 | <c:set var="tituloPagina" value="${empty id ? 'Adicionar item' : 'Editar item'}"/>
004 | <%@ include file="cabecalho.jspf" %>
005 | <a class="back-link" href="${pageContext.request.contextPath}/itens">← Voltar ao catálogo</a>
006 | <div class="page-heading"><p class="eyebrow">CUIDE DA SUA COLEÇÃO</p><h1>${empty id ? 'Uma
nova história.' : 'Cada detalhe importa.'}</h1><p class="subtitle">${empty id ? 'Preencha os dados
para adicionar um item ao seu acervo.' : 'Atualize as informações desta obra.'} Todos os campos são
 obrigatórios.</p></div>
007 | <c:if test="${not empty erros}"><div class="notice error" role="alert"><strong>Revise os
campos indicados.</strong><ul><c:forEach var="erro" items="${erros}"><li><c:out
value="${erro.value}"/></li></c:forEach></ul></div></c:if>
008 | <form class="edit-form" action="${pageContext.request.contextPath}/itens/salvar"
method="post">
009 |   <input type="hidden" name="csrfToken" value="${sessionScope.csrfToken}">
010 |   <c:if test="${not empty id}"><input type="hidden" name="id" value="${id}"></c:if>
011 |   <div class="form-grid">
012 |     <div class="field"><label for="tipoMidia">Tipo de mídia</label><select id="tipoMidia"
name="tipoMidia" required aria-describedby="erro-tipoMidia"><c:forEach var="tipo"
items="${tipos}"><option value="${tipo}" ${form.tipoMidia == tipo.name() ? 'selected' : ''}><c:out
value="${tipo.descricao}"/></option></c:forEach></select><small id="erro-tipoMidia" class="field-
error"><c:out value="${erros.tipoMidia}"/></small></div>
013 |     <div class="field"><label for="anoLancamento">Ano de publicação ou
lançamento</label><input id="anoLancamento" name="anoLancamento" type="number" min="1"
max="${anoMaximo}" required value="${fn:escapeXml(form.anoLancamento)}" aria-describedby="erro-
anoLancamento"><small id="erro-anoLancamento" class="field-error"><c:out
value="${erros.anoLancamento}"/></small></div>
014 |     <div class="field field-wide"><label for="titulo">Título</label><input id="titulo"
name="titulo" maxlength="255" required placeholder="Como se chama esta obra?"
value="${fn:escapeXml(form.titulo)}" aria-describedby="erro-titulo"><small id="erro-titulo"
class="field-error"><c:out value="${erros.titulo}"/></small></div>
015 |     <div class="field"><label for="autorDiretor">Autor ou diretor</label><input
id="autorDiretor" name="autorDiretor" maxlength="255" required
value="${fn:escapeXml(form.autorDiretor)}" aria-describedby="erro-autorDiretor"><small id="erro-
autorDiretor" class="field-error"><c:out value="${erros.autorDiretor}"/></small></div>
016 |     <div class="field"><label for="genero">Gênero</label><input id="genero" name="genero"
maxlength="100" required placeholder="Ex.: Ficção científica" value="${fn:escapeXml(form.genero)}"
aria-describedby="erro-genero"><small id="erro-genero" class="field-error"><c:out
value="${erros.genero}"/></small></div>
017 |     <div class="field field-wide"><label for="sinopse">Sinopse</label><textarea id="sinopse"
name="sinopse" rows="6" maxlength="2000" required aria-describedby="sinopse-ajuda erro-
sinopse"><c:out value="${form.sinopse}"/></textarea><div class="field-help"><span id="sinopse-
ajuda">Um breve resumo, com até 2.000 caracteres.</span><span id="sinopse-contador" aria-
live="polite"></span></div><small id="erro-sinopse" class="field-error"><c:out
value="${erros.sinopse}"/></small></div>
018 |   </div><div class="form-actions"><button class="button" type="submit">${empty id ?
'Adicionar ao acervo' : 'Salvar alterações'} <span aria-hidden="true"> </span></button><a
class="button button-ghost" href="${pageContext.request.contextPath}/itens">Cancelar</a></div>
019 | </form>
020 | <%@ include file="rodape.jspf" %>
```

src/main/webapp/WEB-INF/views/lista.jsp

```
001 | <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
002 | <%@ include file="tags.jspf" %>
003 | <c:set var="tituloPagina" value="Meu catálogo"/>
004 | <%@ include file="cabecalho.jspf" %>
005 | <section class="intro"><div><p class="eyebrow">SUA COLEÇÃO, DO SEU JEITO</p><h1>Uma boa
história<br>merece ser <em>guardada.</em></h1><p class="subtitle">Organize os livros, filmes e
séries que fazem parte do seu mundo.</p></div><div class="intro-mark" aria-
hidden="true"><span>LER.</span><span>VER.</span><span>REVER.</span></div></section>
006 | <section class="catalog-section" aria-labelledby="colecao-titulo">
007 |   <div class="section-top"><div><p class="eyebrow">EXPLORE SEU ACERVO</p><h2 id="colecao-
titulo">${empty termo ? 'Todos os itens' : 'Resultados da busca'} <span
class="count">${fn:length(itens)}</span></h2></div>
008 |     <form class="search" action="${pageContext.request.contextPath}/itens" method="get"
role="search"><label class="sr-only" for="q">Buscar por título ou autor/diretor</label><input
id="q" name="q" maxlength="255" placeholder="Título, autor ou diretor..."
value="${fn:escapeXml(termo)}"><button type="submit">Buscar <span aria-
hidden="true"> </span></button></form>
009 |   </div>
010 |   <c:if test="${not empty termo}"><p class="search-summary">Buscando por “<c:out
value="${termo}"/>”. <a href="${pageContext.request.contextPath}/itens">Limpar busca</a></p></c:if>
011 |   <c:choose><c:when test="${empty itens}"><div class="empty-state"><span class="empty-icon"
aria-hidden="true"> </span><h3>${empty termo ? 'Sua próxima história começa aqui.' : 'Nenhum item
encontrado.'}</h3><p>${empty termo ? 'Adicione seu primeiro livro, filme ou série ao catálogo.' :
'Tente outro título, autor ou diretor.'}</p><a class="button"
href="${pageContext.request.contextPath}/itens/novo">Adicionar item</a></div></c:when>
012 |   <c:otherwise><div class="card-grid"><c:forEach var="item" items="${itens}">
013 |     <article class="media-card"><div class="card-art art-${item.tipoMidia}"><span class="art-
label"><c:out value="${item.tipoMidia.descricao}"/></span><span class="art-letter" aria-
hidden="true"><c:out value="${fn:substring(item.titulo, 0, 1)}"/></span><span class="art-
year">${item.anoLancamento}</span></div><div class="card-body"><p class="card-genre"><c:out
value="${item.genero}"/></p><h3><a
href="${pageContext.request.contextPath}/itens/detalhes?id=${item.id}"><c:out
value="${item.titulo}"/></a></h3><p class="card-author"><c:out value="${item.autorDiretor}"/></p><p
 class="card-synopsis"><c:out value="${item.sinopse}"/></p><a class="card-link"
href="${pageContext.request.contextPath}/itens/detalhes?id=${item.id}">Ver detalhes <span aria-
hidden="true"> </span></a></div></article>
014 |   </c:forEach></div></c:otherwise></c:choose>
015 | </section>
016 | <%@ include file="rodape.jspf" %>
```

src/main/webapp/WEB-INF/views/rodape.jspf

```
001 | <%@ page pageEncoding="UTF-8" %>
002 | </main>
003 | <footer class="container site-footer"><span>Acervo · Histórias que ficam com
você.</span><span>Livros, filmes e séries em um só lugar.</span></footer>
004 | </body></html>
```

src/main/webapp/WEB-INF/views/tags.jspf

```
001 | <%@ taglib prefix="c" uri="jakarta.tags.core" %>
002 | <%@ taglib prefix="fn" uri="jakarta.tags.functions" %>
```

src/main/webapp/assets/app.js

```
001 | // Melhorias progressivas: os formulários e a exclusão funcionam sem JavaScript.
002 | const sinopse = document.querySelector('#sinopse');
003 | const contador = document.querySelector('#sinopse-contador');
004 | if (sinopse && contador) {
005 |   const atualizar = () => { contador.textContent = `${sinopse.value.length}/2000`; };
006 |   sinopse.addEventListener('input', atualizar);
007 |   atualizar();
008 | }
009 | document.querySelector('[data-open-delete]')?.addEventListener('click', () => {
010 |   const painel = document.querySelector('#confirmar-exclusao');
011 |   painel.open = true;
012 |   painel.scrollIntoView({ behavior: 'smooth', block: 'nearest' });
013 |   painel.querySelector('button').focus();
014 | });
```

maven.ps1

```
001 | param([Parameter(ValueFromRemainingArguments = $true)][string[]]$MavenArgs)
002 | $ErrorActionPreference = 'Stop'
003 | Push-Location $PSScriptRoot
004 | try {
005 |     $mavenVersion = '3.9.9'
006 |     $mavenDirectory = Join-Path $PSScriptRoot ".tools/apache-maven-$mavenVersion"
007 |     $mavenExecutable = Join-Path $mavenDirectory 'bin/mvn.cmd'
008 |     $existingMaven = Get-Command mvn.cmd -ErrorAction SilentlyContinue
009 |     if ($existingMaven) { $mavenExecutable = $existingMaven.Source }
010 |     elseif (-not (Test-Path -LiteralPath $mavenExecutable)) {
011 |         $toolsDirectory = Join-Path $PSScriptRoot '.tools'
012 |         New-Item -ItemType Directory -Path $toolsDirectory -Force | Out-Null
013 |         $mavenArchive = Join-Path $toolsDirectory "apache-maven-$mavenVersion-bin.zip"
014 |         $downloadUrl = "https://repo.maven.apache.org/maven2/org/apache/maven/apache-
maven/$mavenVersion/apache-maven-$mavenVersion-bin.zip"
015 |         Write-Host "Baixando Maven $mavenVersion para a pasta local .tools..."
016 |         Invoke-WebRequest -Uri $downloadUrl -OutFile $mavenArchive
017 |         $checksum = ((Invoke-WebRequest -Uri "$downloadUrl.sha512").Content.Trim() -split
'\s+')[0]
018 |         if ((Get-FileHash -LiteralPath $mavenArchive -Algorithm
SHA512).Hash.ToLowerInvariant() -ne $checksum.ToLowerInvariant()) {
019 |             throw 'O SHA-512 do Maven não confere. O arquivo não será executado.'
020 |         }
021 |         Expand-Archive -LiteralPath $mavenArchive -DestinationPath $toolsDirectory -Force
022 |     }
023 |     & $mavenExecutable "-Dmaven.repo.local=$PSScriptRoot/.m2" @MavenArgs
024 |     exit $LASTEXITCODE
025 | } finally { Pop-Location }
```

iniciar.ps1

```
001 | param([switch]$SemCompilar)
002 | $ErrorActionPreference = 'Stop'
003 | Push-Location $PSScriptRoot
004 | try {
005 |     if (-not (Get-Command java -ErrorAction SilentlyContinue)) { throw 'Instale o JDK 17 ou
superior e adicione o Java ao PATH.' }
006 |     if (-not $SemCompilar) {
007 |         & powershell -NoProfile -ExecutionPolicy Bypass -File "$PSScriptRoot/maven.ps1"
package '-DskipTests'
008 |         if ($LASTEXITCODE -ne 0) { throw 'A compilação falhou. Consulte as mensagens acima.'
}
009 |     }
010 |     if (-not (Test-Path -LiteralPath 'target/catalogo/WEB-INF/web.xml')) { throw 'Compile o
projeto antes de usar -SemCompilar.' }
011 |     $runtimeClasspath = 'target/classes' + [IO.Path]::PathSeparator + 'target/dependency/*'
012 |     & java '-Dfile.encoding=UTF-8' -cp $runtimeClasspath
br.edu.catalogo.launcher.ServidorLocal
013 | } finally { Pop-Location }
```